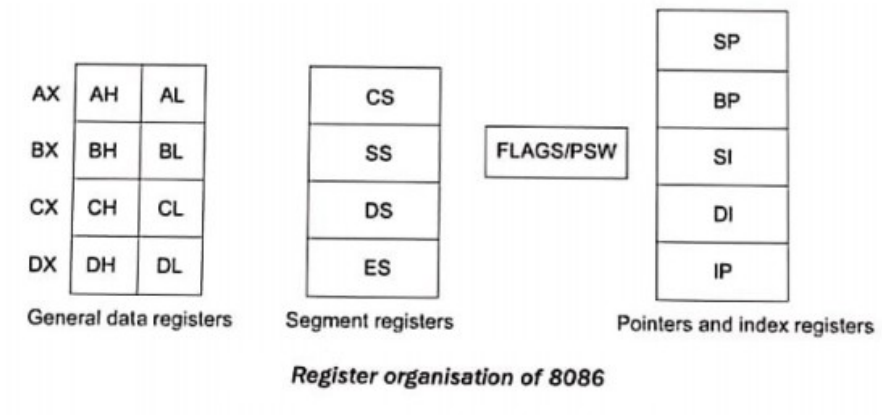
Q. 8086 Programmer's model: Register organization (IMP)



Register organisation of 8086

8086 has a powerful set of registers known as ***general purpose registers*** and ***special purpose registers***. All of them are 16-bit registers.
The 14 registers of 8086 microprocessor are categorized into four groups. They are

1. **General purpose data registers**
2. **Pointer & Index registers**
3. **Segment registers**
4. **Flag register**

General Data Registers:

The registers ***AX, BX, CX* and *DX*** are the general purpose 16-bit registers.

AX – Accumulator
BX – Base (offset register)
CX – Counter
DX – Data register
***AX* is** used as 16-bit **accumulator**. The lower 8-bit is designated as *AL* and higher 8-bit is designated as *AH*. *AL and AH can* be used as an 8-bit accumulator for 8-bit operation.
All data register can be used as either 16 bit or 8 bit.

BX is a 16 bit register, but *BL* indicates the lower 8-bit of *BX* and *BH* indicates the higher 8-bit of *BX*. ***The register BX is*** used as ***offset storage*** for forming physical address in case of certain addressing modes.

The register *CX* is used default ***counter*** in case of string and loop instructions.

***DX* register is a** general purpose register which may be used as an implicit operand or destination in case of a few instructions.

Segment Registers:

There are 4 segment registers. They are:
Code Segment Register (CS)
Data Segment Register (DS)
Extra Segment Register (ES)
Stack Segment Register (SS)

The 8086 architecture uses the concept of **segmented memory**. 8086 able to address a memory capacity of 1megabyte and it is byte organized. This 1 megabyte memory is divided into 16 logical segments. Each segment contains 64 kbytes of memory.

So, the CS, DS, SS and ES segment registers respectively contains the segment addresses for the code, data, stack and extra segments of the memory.

Code segment register (CS): is used for addressing memory location in the code segment of the memory, where the executable program is stored.

Data segment register (DS): points to the data segment of the memory where the data is stored.

Extra Segment Register (ES): also refers to a segment in the memory which is another data segment in the memory.

Stack Segment Register (SS): is used for addressing stack segment of the memory. The stack segment is that segment of memory which is used to store stack data.

While addressing any location in the memory bank, the **physical address which is 20-bit** is calculated from two parts:
Physical address = segment base X 10H + offset address

Segment base: 16-bit content of segment register used to find the memory segment address related to different segment.
The 16 bit contents of the segment register actually point to the starting location of a particular segment.
Segment base X 10H = **Starting address of the 64KB segment**

Offset address:
The second part is the offset value in the offset registers point the address within the 64KB segment. To address a specific memory location within a segment, we need an offset address.

The offset address is also 16 bit long so that the maximum offset value can be FFFFH, and the maximum size of any segment is thus 64 K locations.

Address within the segment = 20 bit physical address of the memory bank= Segment base X 10H + offset
Minimum value of offset = 0000H
Maximum value of offset = FFFFH

Segment base X 10H + Maximum value of offset= **Last address of the 64KB segment**

The advantage of segmentation is that instead of maintaining a 20 bit register for a physical address, the processor just maintains two 16 bit registers which are within the word length capacity of the machine.

Pointers and Index Registers:

The index and pointer registers are given below:

IP—Instruction pointer
BP—Base pointer
SP—Stack pointer
SI—Source index
DI—Destination index

The **pointers registers** contain offset within the particular segments.

IP—It is a 16-bit register which always points to the next instruction to be executed within the currently executing **code segment**. So, this register contains the 16-bit offset address pointing to the next instruction code within the 64kB of the code segment area. Its content is automatically incremented as the execution of the next instruction takes place.

SP-This is the stack pointer. It is of 16 bits. It points to the topmost address of the stack memory. It's offset address relative to **stack segment.**

BP-This is the base pointer. It is of 16 bits. It is primary used in accessing parameters passed by the stack. It's offset address relative to **stack segment.**

The **index registers** (SI and DI) are used as general purpose registers as well as for offset storage in case of indexed, base indexed and relative base indexed addressing modes.
The **register *SI* is** used to store the offset of source data in **data segment**.
The register ***DI*** is used to store the offset of destination in data or **extra segment**.
The index registers are particularly useful for string manipulation.

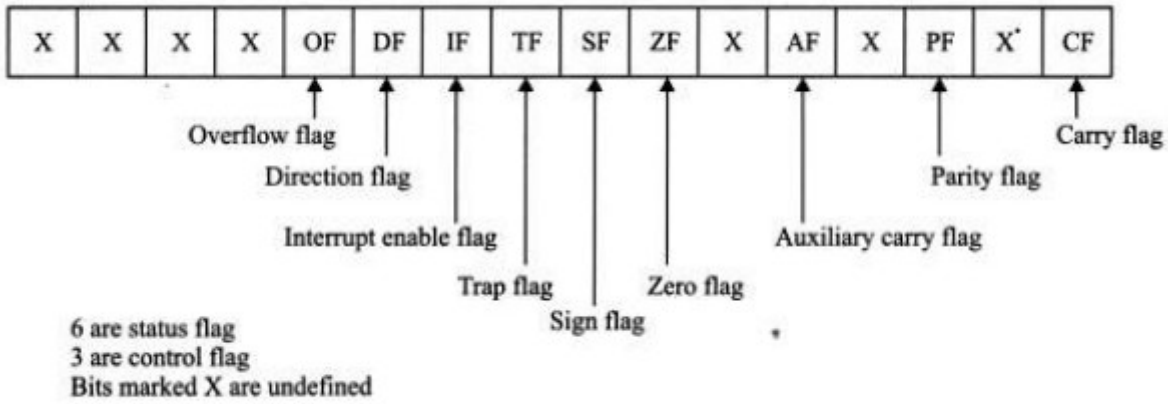
Segment and its default offset

Segment	Offset Registers	Function
CS	IP	Address of the next instruction
DS	BX, DI, SI	Address of data
SS	SP, BP	Address in the stack
ES	DI	Address of destination data (for string operations)

8086 flag register and its functions:

It is a 16-bit register, also called Program Status Word (PSW). There are total 9 flags in the flag register and these are divided into two groups:

- Status flags
- Control flags



FLAG REGISTER FORMAT

Status flags

The status flags are set/reset depending on the results of some arithmetic or logical operations during program execution.
The 6 status flags are:

1. Sign Flag (S)
2. Zero Flag (Z)
3. Auxiliary Cary Flag (AC)
4. Parity Flag (P)
5. Carry Flag (CY)
6. Overflow Flag (OV)

Sign Flag (S):

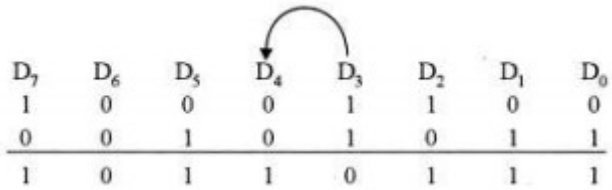
After 8-bit or 16-bit operation if the MSB of the result is 1, Sign flag=1 else it is zero.
Basically it is used for signed Arithmetic operation to indicate whether result of operation is negative or positive.
If the SF=1(set), result is negative.
If the SF=0 (reset), result is positive.
For unsigned operation all the bits of the result are magnitude there is no sign bit, so ignore sign flag value.

Zero Flag (Z):

After any arithmetical or logical operation if the result is zero, the zero flag becomes set i.e. 1, otherwise it becomes reset i.e. 0.

Auxiliary Cary Flag (AF):

This flag is used in BCD number system (0-9).
AF= 1 (set), if carry generate from D3 bit (lower nibble) to D4(higher nibble) bit during ***8-bit addition*** and it is reset or zero when there is no carry.
This is the only flag which is not accessible by the programmer



Parity Flag (P)

It is set to 1 if result of byte operation or lower byte of the word operation contain even number of ones; otherwise it is reset.
P=1, indicate result is even parity
P=0, indicate result is odd parity

Carry Flag (CY)

This flag is set, when there is a carry out of MSB in case of addition or a borrow into the MSB in case of subtraction.
During subtraction (A-B), if A>B it becomes reset and if (A<B) it becomes set. Carry flag is also called borrow flag.
1-carry out from MSB bit on addition or borrow into MSB bit on subtraction
0-no carry out or borrow into MSB bit.

Overflow Flag (OV)

This flag is used in signed arithmetic operation.

If the result of a signed operation is large enough to accommodate in a destination register then this flag will be set otherwise it will be cleared.

The result is of more than 7-bits in size in case of 8-bit signed operation and more than 15-bits in size in case of 16-bit sign operations, and then the overflow will be set.

For addition overflow flag is set, if there is a carry out from the MSB and no carry to the MSB or vice versa, else it is reset.

For subtraction, it is set when the MSB needs a borrow and there is no borrow from the MSB, or vice versa.

EXAMPLE-1

Give the contents of the flag register after execution of following addition.

```
  0110 0101 1101 0001
+ 0010 0011 0101 1001
  ------------------
  1000 1001 0010 1010
```

Solution : SF = 1, ZF = 0, PF = 1, CF = 0, AF = 0 , OF = 1

EXAMPLE-2

Give the contents of the flag register after execution of following subtraction

```
  0110 0111 0010 1001
- 0011 0101 0100 1010
  ------------------
  0011 0001 1101 1111
```

Solution : SF = 0, ZF = 0, PF = 1, CF = 0, AF = 1, OF = 0

EXAMPLE-3

Assume the content of **AX=F345H** and **BX=089AH**.

After execution of **ADD AX, BX**, what is the content of status flags?

```
1 1 1 1 0 0 1 1 0 1 0 0 0 1 0 1
0 0 0 0 1 0 0 0 1 0 0 1 1 0 1 0
---------------------------------
1 1 1 1 1 0 1 1 1 1 0 1 1 1 1 1
```

Solution: SF= 1, ZF=0, PF=0, CF=0, AF=0, OF=0

Control Flags – The control flags enable or disable certain operations of the microprocessor. There are 3 control flags in 8086 microprocessor and these are:

1. **Directional Flag (DF) –** This flag is used by string manipulation instructions.
   If this flag bit is '0', the string is processed beginning from the lowest address to the highest address, i.e. **auto-incrementing mode.** Otherwise, the string is processed from the highest address towards the lowest address, i.e. **auto-decrementing mode**.
   Instruction to set DF –STD
   Instruction to reset DF-CLD
2. **Interrupt Flag (IF) –**
   This flag is to mask or un-mask maskable interrupt (INTR).
   IF=1, enable INTR signal so the microprocessor will recognize interrupt requests from the peripherals.

IF=0, disable (mask) INTR signal so the microprocessor will not recognize any interrupt requests and will ignore them.

Instruction to set IF-STI

Instruction to reset IF-CLI

3. **Trap Flag (T) –** one way to debug a program is to run the program one instruction at a time and see the contents of used registers and memory variables after execution of every instruction. This process is called **'single stepping'** through a program. If set, a trap is executed after execution of each instruction, i.e. interrupt service routine is executed which displays various registers and memory variable contents on the display after execution of each instruction. Thus programmer can easily trace and correct errors in the program.

   TF=1, Enable single step execution mode

   TF=0, disable it